



HCMUS
Viet Nam National University
Ho Chi Minh City
University of Science

VNUHCM - UNIVERSITY OF SCIENCE

Faculty of Information Technology

Course: Introduction to Artificial Intelligence

LAB 1 - SEARCHING

SAIGON ROUTE LAB

Tourist Route Planner for Visiting Multiple Landmarks
in Ho Chi Minh City

Group ID: 1

24127052 Phùng Bảo Khang
24127069 Thái Kiệt
24127167 Nguyễn Đăng Hậu
24127257 Nguyễn Thành Trung
24127378 Nguyễn Huy Hoàng

Ho Chi Minh City, August 2026

Contents

1	Group Introduction and Completion	1
1.1	Member responsibilities	1
1.2	Requirement completion	1
2	Problem Context	2
3	Problem Modeling	2
4	Dataset	3
5	Cost Function and Traffic Conditions	3
6	Algorithm Principles	4
6.1	Breadth-First Search	5
6.2	Depth-First Search	5
6.3	Uniform Cost Search	5
6.4	A* Search	5
6.5	Dijkstra's Algorithm	5
6.6	Greedy Best-First Search	5
6.7	Theoretical comparison	6
7	Multi-Location Optimization	6
8	Program Flow and Architecture	6
9	Experimental Evaluation	7
9.1	Algorithm comparison	8
9.2	Traffic-profile effect	8
9.3	Multi-location comparison	8
10	GUI, Route Explanation, and API	9
11	Installation and Use	11
12	Verification	12
13	Limitations and Future Work	12
14	Conclusion	12
15	Use of AI Tools	13
16	Acknowledgement	13

1 Group Introduction and Completion

1.1 Member responsibilities

Table 1 records the principal responsibility assigned to each member. Every listed work package is implemented and integrated in the final application; completion denotes the assigned work, not a percentage of total individual authorship.

Table 1: Group members, responsibilities, and completion.

Student ID	Full name	Main contribution	Done
24127378	Nguyễn Huy Hoàng	BFS, Vietnamese traffic context, report introduction	100%
24127167	Nguyễn Đăng Hậu	DFS, graph model, dataset, and cost function	100%
24127257	Nguyễn Thành Trung	UCS, route explanation, and algorithm analysis	100%
24127052	Phùng Bảo Khang	A^* , heuristic, GUI, and visualization	100%
24127069	Thái Kiệt	Dijkstra, Greedy Best-First, and multi-location optimization	100%

1.2 Requirement completion

Table 2: Implementation evidence for the Lab 1 requirements.

Requirement	Evidence	Status
Vietnamese traffic scenario	24 HCMC landmarks and OSM-derived roads	Complete
Graph, dataset, and cost	481 nodes, 995 directed edges, weighted traffic cost	Complete
BFS, DFS, UCS, and A^*	Independent implementations with shared contracts	Complete
Two additional algorithms	Dijkstra and Greedy Best-First	Complete
Multi-location planning	Nearest Neighbor and Exact Brute Force	Complete
GUI and visualization	React, Leaflet, REST, and WebSocket step stream	Complete
Explanation and alternatives	Named roads, congestion, guarantee, and baseline comparison	Complete
Verification	70 tests, all-pair audit, and frontend production build	Complete

2 Problem Context

Tourists often want to visit several attractions in central Ho Chi Minh City in one trip. The physically shortest route may be unsuitable because major roads can be slow during rush hour, rain can increase travel time and road risk, and one-way streets constrain movement. The project therefore recommends routes using distance, estimated travel time, congestion, and risk instead of distance alone.

The application supports two related tasks:

1. find a route from one selected landmark to another;
2. choose an efficient visiting order and complete route for several required landmarks.

This is a Vietnamese traffic problem rather than an abstract maze. Landmark names, road geometry, road classes, and directionality are grounded in Ho Chi Minh City data.

3 Problem Modeling

The traffic network is a directed weighted graph $G = (V, E)$.

- **State:** the current routable road node.
- **Initial state:** the unique road node assigned to the selected starting landmark.
- **Goal test:** the current node equals the destination node.
- **Action:** follow one outgoing directed road edge.
- **Path:** an ordered node sequence connected by valid edges.
- **Path cost:** the sum of weighted edge costs.

Each edge stores physical distance, estimated travel time, congestion, risk, road type, road name, one-way metadata, and drawing geometry. Algorithms never infer a reverse edge: reverse travel is possible only when the loader provides that directed edge.

Table 3: Runtime edge fields.

Field	Unit or range	Meaning
distance_km	km, non-negative	Physical road length
time_min	minutes, non-negative	Estimated travel time
congestion	1 to 5	Scenario traffic level
risk	0 to 5	Road-class and weather penalty
road_type	text	OpenStreetMap highway class
oneway	Boolean metadata	Whether reverse travel is prohibited
geometry	coordinate list	Line drawn by Leaflet

4 Dataset

The source GeoJSON was produced from OpenStreetMap using the group’s OSMnx tooling. The extracts contain 1,075 directed road features, 519 intersection records, and approximately 95.083 km of road geometry. The loader normalizes duplicate endpoint pairs, joins two adjacent major components, and retains the largest strongly connected graph.

Table 4: Final graph dimensions.

Property	Value
Routable nodes	481
Directed edges	995
One-way directed edges	403
Tourist landmarks	24
Unique assigned landmark nodes	24
Traffic profiles	3

Every landmark is assigned to a unique nearby routable node so that two required stops cannot collapse into one visit. Marker-to-node offsets are shown as dashed access lines in the GUI. The mean offset is about 100 m and the maximum is 383.5 m for Ben Thanh Market. These access offsets are not included in road-route totals and remain a documented limitation.

The two source boundaries are separately clipped. One bidirectional boundary connection, represented by two directed simulated edges, joins their major road components. The GUI identifies this assumption as *Boundary connector*.

The 24 landmarks include Ben Thanh Market, Independence Palace, War Remnants Museum, Notre Dame Cathedral, Central Post Office, Nguyen Hue Walking Street, Bach Dang Wharf, Saigon Opera House, Tao Dan Park, Fine Arts Museum, Tan Dinh Church, Saigon Zoo, Vinh Nghiem Pagoda, and other central destinations. The complete machine-readable list is stored in `data/landmarks.json`.

5 Cost Function and Traffic Conditions

For an edge e , the configurable weighted cost is

$$C(e) = \alpha d(e) + \beta t(e) + \gamma q(e) + \delta r(e), \quad (1)$$

where d is distance in kilometres, t is estimated time in minutes, q is congestion, and r is risk. Every term and weight is non-negative; therefore, UCS and Dijkstra satisfy their non-negative-cost optimality condition.

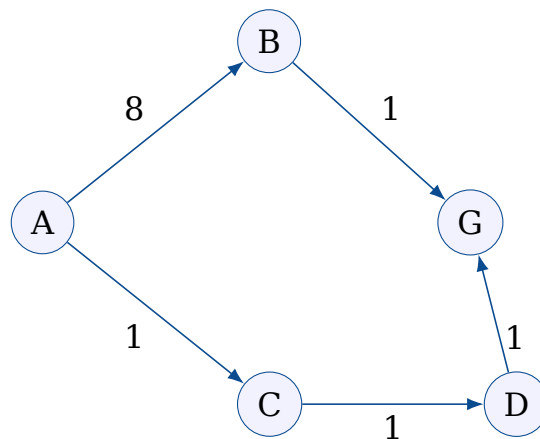
Table 5: Cost-criterion presets.

Preset	α	β	γ	δ	Purpose
Balanced	1.00	0.40	0.08	0.12	Trade off all four factors
Fastest	0.20	1.20	0.04	0.08	Emphasize estimated time
Shortest	1.50	0.05	0.01	0.03	Emphasize distance
Low congestion	0.70	0.30	0.25	0.15	Avoid congested roads
Low risk	0.50	0.30	0.08	0.80	Avoid higher-risk roads

The **normal** profile uses base road-class values. **Rush hour** increases time, congestion, and risk, with a larger time multiplier on primary and secondary roads. **Rainy** increases time, congestion, and risk using a deterministic hybrid scenario. These are reproducible academic scenarios, not live sensor measurements.

6 Algorithm Principles

The group designed the graph in Figure 1 for unit tests and step-by-step video explanation. Route $A \rightarrow B \rightarrow G$ has two edges and cost 9, while $A \rightarrow C \rightarrow D \rightarrow G$ has three edges and cost 3. The example exposes the difference between structural, cost-aware, and heuristic search.

**Figure 1:** Group-designed graph with competing minimum-edge and minimum-cost routes.**Table 6:** Expansion order and result on the group-designed graph.

Algorithm	Expansion or pop order	Final route	Cost
BFS	A, B, C, G	A -> B -> G	9
DFS	A, B, G	A -> B -> G	9
UCS	A(0), C(1), D(2), G(3) by g	A -> C -> D -> G	3
A^*	A(2), C(3), D(3), G(3) by $f = g + h$	A -> C -> D -> G	3
Dijkstra	A(0), C(1), D(2), G(3) by distance label	A -> C -> D -> G	3
Greedy	A(2), B(0), G(0) by h	A -> B -> G	9

6.1 Breadth-First Search

BFS uses a FIFO queue and expands the shallowest discovered node. It is complete on this finite graph and optimal only by edge count when every action has equal cost. It does not generally minimize the traffic cost in Equation 1.

6.2 Depth-First Search

DFS uses a LIFO stack and follows one branch before backtracking. A discovered set prevents cycles. It is complete for this finite explicit graph but does not guarantee a short, fast, or low-cost route.

6.3 Uniform Cost Search

UCS orders the frontier by accumulated cost $g(n)$. It stops only when the goal is removed from the priority queue with its best cost. With non-negative edge costs, it is complete and optimal. The project contains a goal-directed UCS implementation with its own trace.

6.4 A^* Search

A^* orders nodes by

$$f(n) = g(n) + h(n), \quad h(n) = \alpha \text{HaversineDistance}(n, \text{goal}). \quad (2)$$

Haversine distance never exceeds road distance, and every other cost term is non-negative. The heuristic is therefore admissible. It is also consistent: the metric triangle inequality and the edge cost lower bound imply $h(n) \leq C(n, n') + h(n')$. Exhaustive checks over every graph edge, all 24 landmark goals, and all five presets found zero consistency violations.

6.5 Dijkstra's Algorithm

Dijkstra also expands the lowest accumulated-cost node. In addition to point-to-point routing, its module exposes a single-source helper used to cache pairwise paths for multi-location optimization. It is complete and optimal for non-negative edge costs.

6.6 Greedy Best-First Search

Greedy orders the frontier only by $h(n)$. It may expand far fewer nodes but ignores accumulated traffic cost when deciding where to explore. A closed set ensures termination on the finite graph; route optimality is not guaranteed.

6.7 Theoretical comparison

Table 7: Theoretical properties used in the application.

Algorithm	Worst-case time	Memory	Complete here	Cost-optimal
BFS	$O(V + E)$	$O(V)$	Yes	Only by edge count
DFS	$O(V + E)$	$O(V)$	Yes, finite graph	No
UCS	$O((V + E) \log V)$	$O(V)$	Yes	Yes, non-negative cost
A^*	$O((V + E) \log V)$ graph bound	$O(V)$	Yes	Yes, admissible heuristic
Dijkstra	$O((V + E) \log V)$	$O(V)$	Yes	Yes, non-negative cost
Greedy	$O((V + E) \log V)$	$O(V)$	Yes, finite graph	No

7 Multi-Location Optimization

Pairwise shortest paths between important nodes are computed with cached single-source Dijkstra runs.

- **Nearest Neighbor** repeatedly visits the currently cheapest unvisited waypoint. It is deterministic and efficient but approximate.
- **Exact Brute Force** evaluates every waypoint permutation when the count is at most eight. It guarantees the best ordering for the reduced pairwise shortest-path problem and has factorial complexity.

Both methods support a fixed final landmark and returning to the start. Duplicate waypoints are removed in input order, and an unreachable mandatory segment produces an explicit failure result. For k required waypoints, Nearest Neighbor needs $O(k^2)$ route-choice work after pairwise preprocessing, whereas Exact Brute Force evaluates $k!$ orders.

8 Program Flow and Architecture

Figure 2 summarizes the separation between the React GUI, FastAPI service, search modules, cost profiles, and OSM graph.

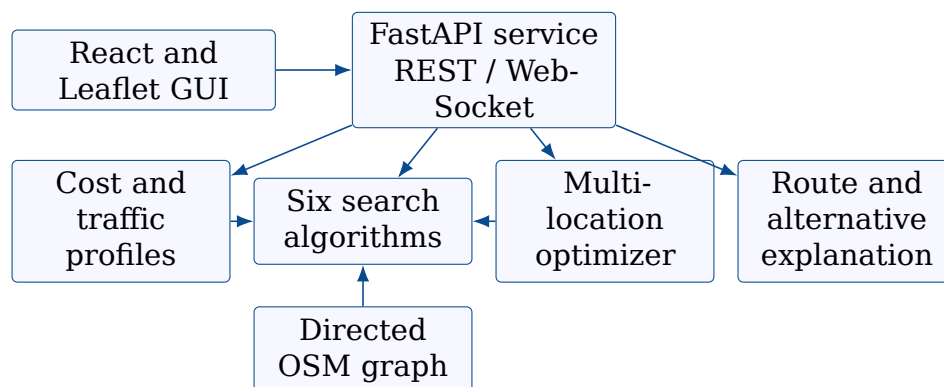


Figure 2: Application architecture and principal data flow.

The common point-to-point search flow is shown in Figure 3. Every expansion can be serialized as a shared trace step containing the current node,

frontier, visited set, parent update, and relevant g , h , and f values.

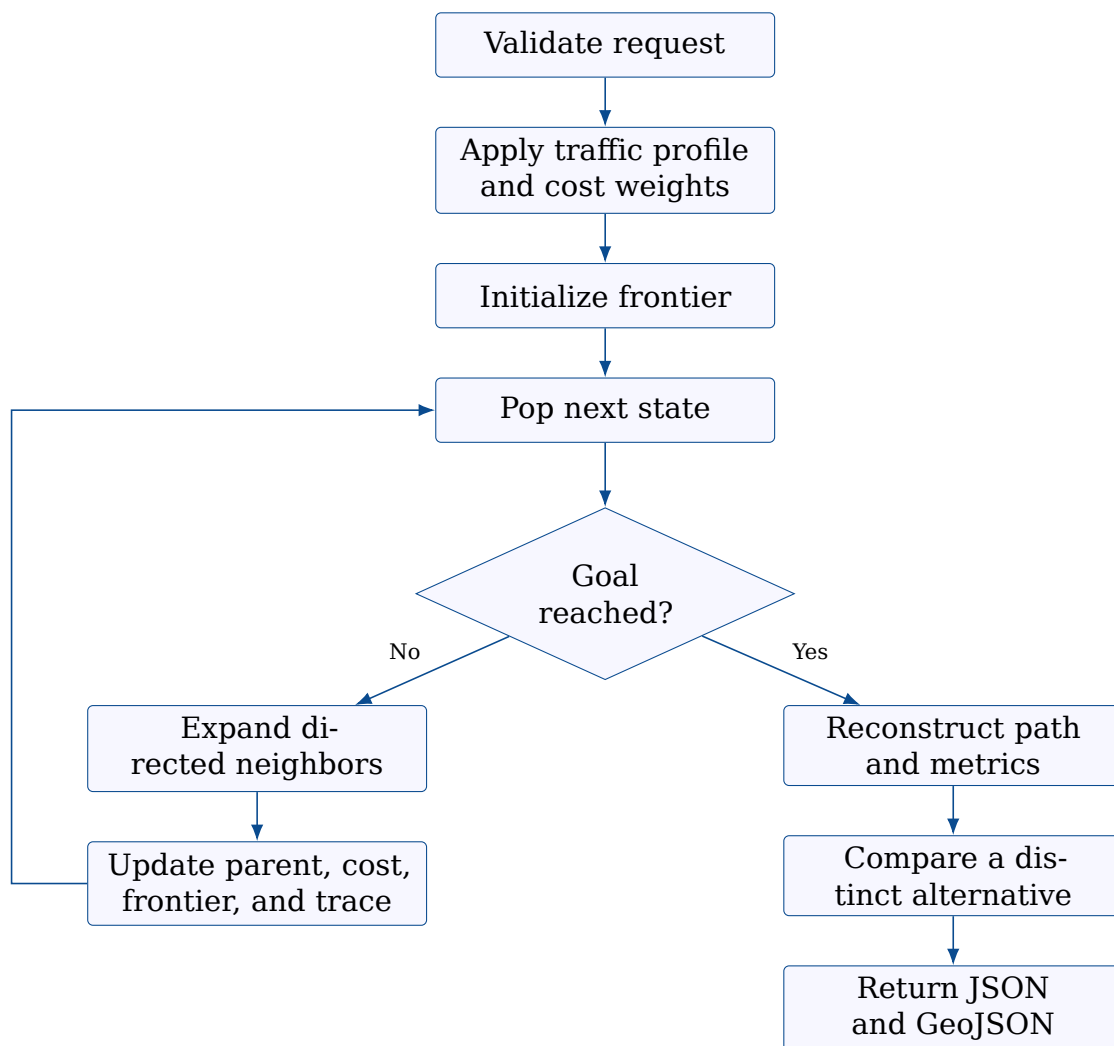


Figure 3: Shared search request and result flow.

The backend has three principal module groups:

- graph loading: `core/osm_loader.py`;
- contracts: `core/contracts.py`;
- search and planning: `algorithms/` and `core/multi_location.py`;
- orchestration: `core/service.py` and `core/explanation.py`;
- API entry point: `app/main.py`.

The GUI consumes the shared API payloads and does not reimplement search logic.

9 Experimental Evaluation

The committed benchmark command is

```
python examples\run_benchmark.py --repeats 10
```

Runtime is the median of ten in-process runs on the development machine. It is useful for controlled comparison but is not a universal performance claim.

9.1 Algorithm comparison

The controlled scenario is Notre Dame Cathedral to Saigon Zoo, balanced cost, and normal traffic.

Table 8: Measured point-to-point algorithm results.

Algorithm	Cost	km	min	Expanded	Generated	Median ms
BFS	7.661	2.046	2.977	206	226	0.890
DFS	35.768	7.598	9.733	150	183	0.982
UCS	7.661	2.046	2.977	218	244	0.294
A*	7.661	2.046	2.977	172	192	0.570
Dijkstra	7.661	2.046	2.977	218	244	0.292
Greedy	7.738	1.422	1.942	19	37	0.135

UCS, A*, and Dijkstra agree on the minimum weighted cost. BFS happens to reach the same route in this case, but that is not its general guarantee. A* expands 46 fewer nodes than UCS or Dijkstra. Greedy expands only 19 nodes and finds a shorter and faster physical route, but its balanced weighted cost is higher. DFS returns a much longer and more expensive route.

9.2 Traffic-profile effect

The scenario in Table 9 is Ben Thanh Market to Independence Palace using Dijkstra and balanced weights.

Table 9: Effect of deterministic traffic profiles.

Profile	Cost	km	min	Nodes	Route effect
Normal	9.365	2.140	3.223	21	Baseline route
Rush hour	11.834	2.140	5.394	21	Same roads, higher penalties
Rainy	13.184	2.145	4.409	20	Changes via Ly Tu Trong and Hai Ba Trung

Rush hour increases estimated time by about 67% while keeping the selected path in this case. Rain changes the route because risk and time penalties alter edge priorities. Across all ordered landmark pairs, 177 of 552 pairs change path in at least one traffic profile.

9.3 Multi-location comparison

The start is Notre Dame Cathedral. The requested waypoints are Ben Thanh Market, Nguyen Hue Walking Street, Bach Dang Wharf, and Fine Arts Museum.

Table 10: Original, nearest-neighbor, and exact visit orders.

Method	Cost	km	min	Gap vs exact
Original input order	12.260	2.689	3.816	9.24%
Nearest Neighbor	13.565	3.194	5.748	20.88%
Exact Brute Force	11.223	2.552	3.647	0%

This case intentionally demonstrates that a local nearest choice can be worse than the original order and the exact order. Exact brute force improves cost by about 8.46% relative to the requested order. The result is optimal for the reduced pairwise problem, not for an unrestricted physical traveling-salesperson model.

- Nearest order: Nguyen Hue Walking Street, Ben Thanh Market, Fine Arts Museum, Bach Dang Wharf.
- Exact order: Nguyen Hue Walking Street, Fine Arts Museum, Ben Thanh Market, Bach Dang Wharf.

10 GUI, Route Explanation, and API

The web GUI supports **Single**, **Compare**, and **Multi** modes. Users choose start, destination, optional waypoints, algorithm, cost criterion, and traffic profile. Leaflet displays the road graph, selected route, landmark access offsets, visited nodes, frontier nodes, and current node.

Single results include graph-node path, named roads, distance, time, cost, runtime, expanded and generated counts, high-congestion roads, optimality, and a distinct baseline comparison. Trace controls expose frontier and visited counts together with g , h , f , and updated cost. Compare mode runs all six algorithms under identical settings. Multi mode shows requested and optimized orders plus the Nearest Neighbor and Exact metrics and gap.

Table 11: Application endpoints.

Endpoint	Purpose
GET /api/health	Health and graph dimensions
GET /api/bootstrap	Landmarks, controls, coordinates, and metadata
GET /api/network	Routable roads as GeoJSON
POST /api/search	Point-to-point result and alternative
POST /api/compare	Six algorithms under one scenario
POST /api/multi-route	Multi-landmark optimization
WS /ws/search	Live search steps followed by final output

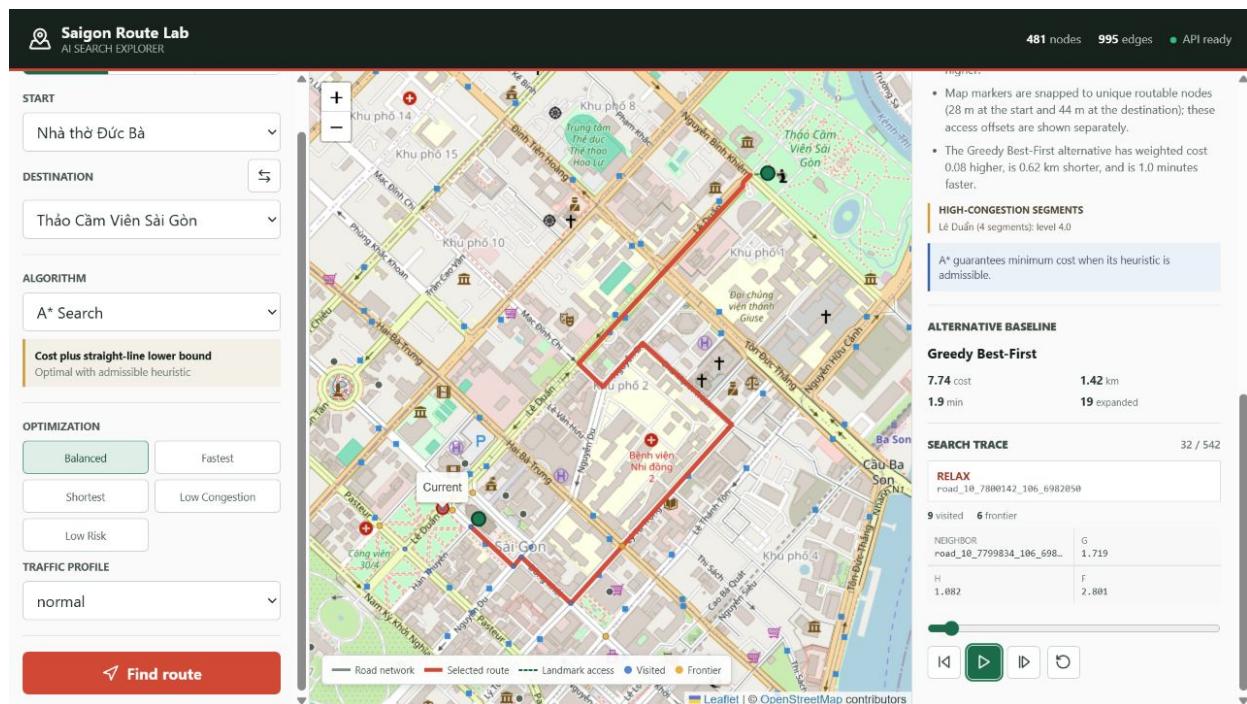


Figure 4: Single search with route explanation and $g/h/f$ trace detail.

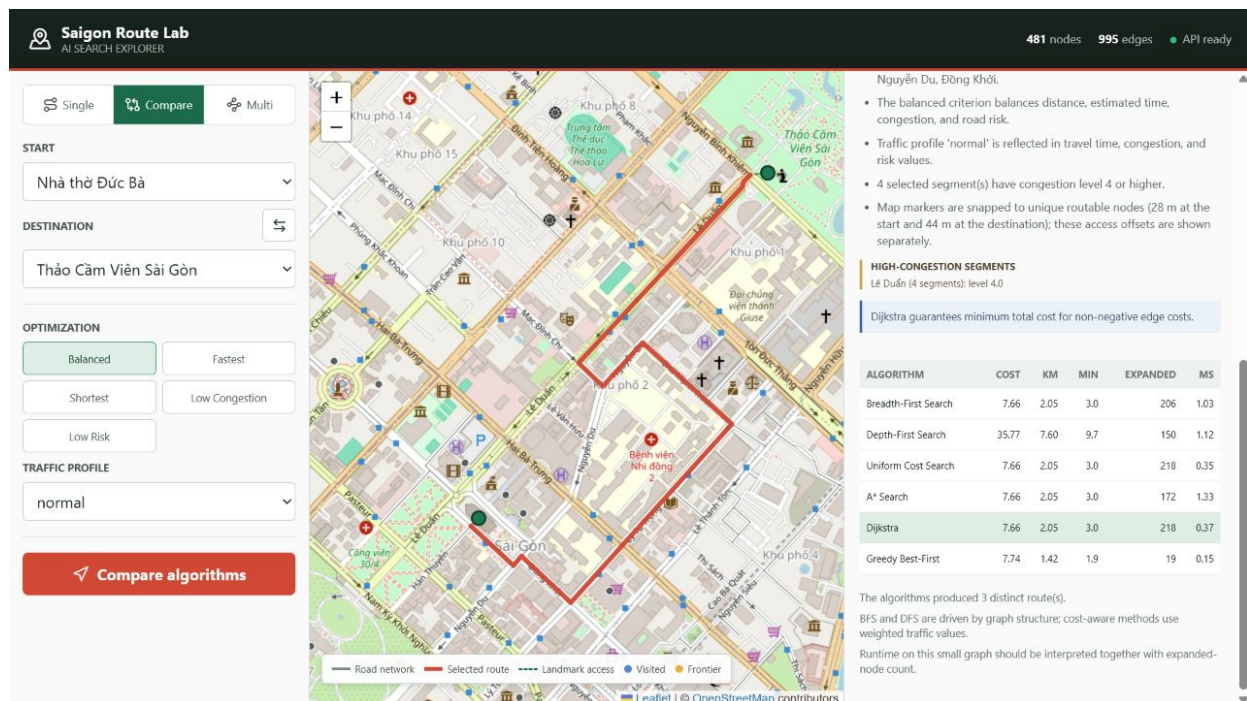


Figure 5: Controlled comparison of BFS, DFS, UCS, A*, Dijkstra, and Greedy.

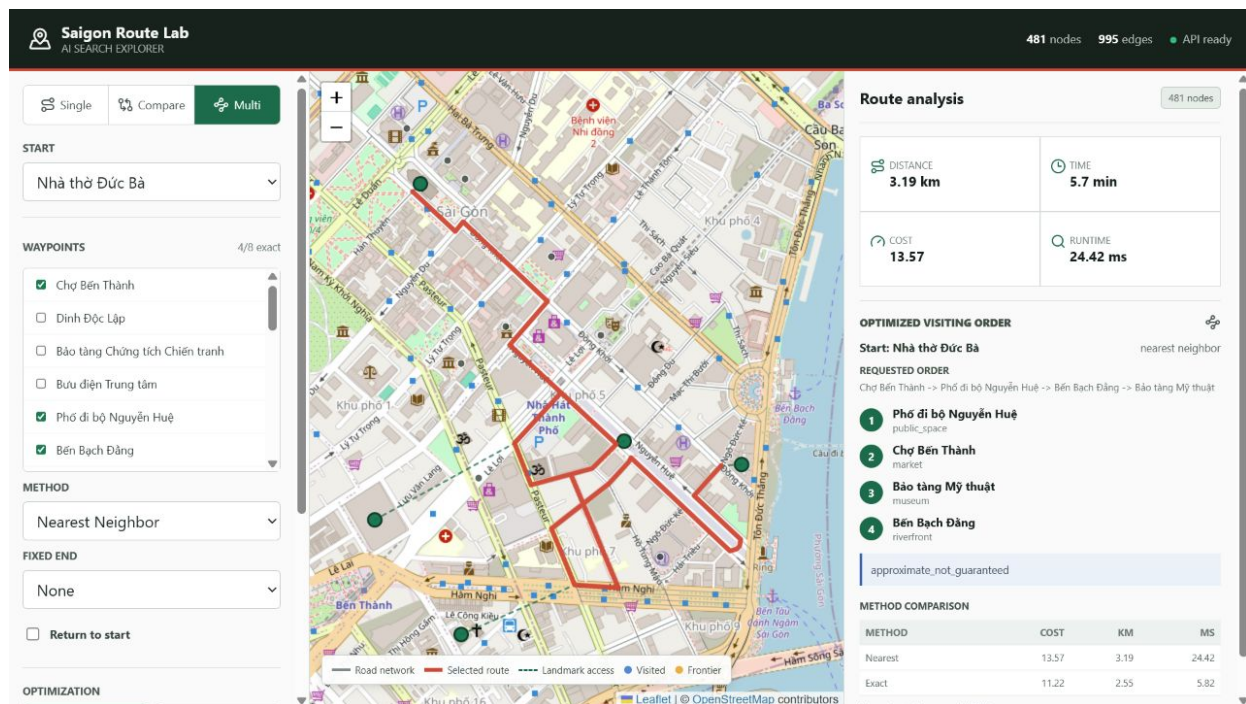


Figure 6: Multi-landmark optimization; nearest-neighbor gap is 20.88%.

11 Installation and Use

Start the backend in one terminal:

```
cd lab-1-backend
python -m venv .venv
.\venv\Scripts\Activate.ps1
python -m pip install -e .
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Start the frontend in a second terminal:

```
cd lab-1-frontend
pnpm install
pnpm dev
```

Open <http://127.0.0.1:5173>. For a first example, choose Notre Dame Cathedral, Saigon Zoo, Dijkstra, balanced cost, and normal traffic. Use the trace controls to inspect expansion steps, Compare for all six algorithms, and Multi for several required landmarks.

12 Verification

The final verification includes:

- 70 passing backend unit and integration tests;
- REST validation and WebSocket streaming coverage;
- 24 unique landmark-to-node assignments;
- all 552 ordered landmark pairs reachable;
- UCS, A^* , and Dijkstra agreeing on optimal cost for all 552 pairs;
- zero A^* heuristic consistency violations over all graph edges, 24 landmark goals, and five cost presets;
- strict TypeScript compilation and a successful Vite production build;
- desktop and mobile browser checks for Single, Compare, and Multi with a clean browser console;
- a clean submodule clone followed by all backend tests and frontend installation and production build.

The exhaustive graph claims are reproducible in the backend repository with:

```
python examples\run_full_audit.py
```

13 Limitations and Future Work

- Traffic and risk are deterministic scenarios rather than live data.
- Six landmarks are more than 100 m from their assigned road node; the maximum access offset is 383.5 m.
- One transparent simulated boundary link joins separate source extracts.
- Exact multi-location search has factorial growth and is capped at eight stops.
- Runtime varies by hardware and browser load.
- The road model does not yet include turn restrictions, traffic signals, vehicle classes, or multiple vehicles.

Future work should use one continuous OSM extract covering every landmark, integrate live traffic and flooding feeds, project landmarks to actual road edges, support travel modes and turn restrictions, and evaluate a larger reproducible scenario suite.

14 Conclusion

Saigon Route Lab applies six search strategies to a real Vietnamese map context under one shared graph and cost model. The implementation demonstrates the trade-off between uninformed, cost-aware, and heuristic search; explains routes and alternatives; and extends point-to-point search to exact and approximate multi-landmark planning. Tests, audits, and benchmarks support these claims while the documented limitations keep them precise.

15 Use of AI Tools

The group used ChatGPT and Codex as supporting tools for organizing the rubric, reviewing implementation ideas, generating candidate test cases, improving technical writing, and checking presentation and report layout. Algorithm behavior, benchmark values, API results, test outcomes, and generated artifacts were verified against the source code and executable application. The group is responsible for reviewing the final submission and explaining every algorithm and design decision during the demonstration.

16 Acknowledgement

The group thanks the lecturer and teaching assistants for the Lab 1 assignment and guidance. OpenStreetMap contributors provide the road data used in the project. The report also acknowledges the software documentation and references listed below.

References

- [1] Course teaching team, *Lab 1 - Searching*, Introduction to Artificial Intelligence course material, 2026.
- [2] Group 1, *Tourist Route Planner for Visiting Multiple Landmarks in Ho Chi Minh City*, project specification supplied with the assignment, 2026.
- [3] OpenStreetMap contributors, *Copyright and License*.
<https://www.openstreetmap.org/copyright>.
- [4] G. Boeing, *OSMnx: Python for Street Networks*.
<https://osmnx.readthedocs.io/>.
- [5] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021.
- [6] VNUHCM – University of Science, *Official Logo and Brand Identity*.
<https://en.hcmus.edu.vn/academics-2/logo-3/>.